14. For

Created by Li Huanyu, last modified by suncaper on Mar 09, 2026

14. For Loops: Iterating Over Sequences

Introduction

In the previous lesson, we used **While Loops** to handle "unknown" durations, like waiting for a file to arrive in an S3 bucket. However, in the world of Data Engineering, we usually work with **known collections**: a list of 500 file paths, a dictionary of configuration settings, or a specific range of dates.

A **For Loop** is used to iterate over a sequence (like a List, Tuple, Dictionary, Set, or String). It is the most common way to process data because it has a natural end point—it stops automatically once it reaches the last item in your collection.

In this lesson, we will cover:

- Basic `for` loop syntax for Lists.
- Using `range()` for counter-based loops.
- Iterating through **Dictionaries** (essential for JSON parsing).
- Why `for` loops are the "single-node" conceptual foundation of Apache Spark.

Part 1: Basic Iteration (Lists)

Python's `for` loop is intuitive. You don't need to manually manage an index counter (like `i = 0`). You simply tell Python: "For every individual item in this collection, do something."

```
# A list of landing zones for our data pipeline
storage_buckets = ["us-east-1", "eu-west-1", "ap-south-1"]

# Python automatically assigns each item to the 'bucket' variable one by one
for bucket in storage_buckets:
    print("Deploying pipeline infrastructure to: " + bucket)
```

Part 2: Generating Sequences with `range()`

Sometimes you don't have a list ready, but you know exactly how many times you want to run a task. The `range()` function generates a sequence of numbers on the fly.

- `range(5)` → Generates 0, 1, 2, 3, 4
- `range(2, 6)` → Generates 2, 3, 4, 5
- `range(0, 10, 2)` → Generates 0, 2, 4, 6, 8 (Starts at 0, stops before 10, jumps by 2)

```
# Simulating a data backfill for the last 3 days
for day in range(1, 4):
    print("Backfilling historical data for: " + str(day) + " day(s) ago...")
```

Part 3: Looping through Dictionaries

When you receive a JSON payload from an API (which becomes a Dictionary in Python), you often need to loop through it to validate keys or extract specific values.

To get both the Key and the Value at the same time, we use the `.items()` method, which unpacks them into two variables.

```
# A dictionary representing a database schema
schema_config = {"id": "INT", "username": "VARCHAR", "is_active": "BOOLEAN"}

# Unpacking the Key (column) and Value (dtype)
for column, dtype in schema_config.items():
    print("Validating Column: [" + column + "] -> Expected Type: " + dtype)
```

Part 4: The "Big Data" Context (Why do we care?)

- **Batch File Processing:** If you have a directory full of 1,000 CSV files, a `for` loop is how you tell Python to "Open file 1, clean it; Open file 2, clean it..." until the folder is completely processed.
- **Schema Evolution:** When moving data from a source database to a data warehouse, you use `for` loops to iterate through the columns of a table to ensure they match the destination's required data types.
- **The Foundation of Spark:** In PySpark, when you use a `map()` or `foreach()` transformation, the Spark engine is essentially running a highly optimized, distributed `for` loop across hundreds of servers simultaneously. Mastering the logic of a loop on a single machine is the prerequisite to mastering it on a cluster.

Part 5: Hands-on Exercise (Batch Data Scrubbing)

Let's simulate a batch process that cleans a list of messy, raw transaction statuses. We need to capitalize the text for our database, but we want to completely skip any records marked as "INTERNAL".

1. Open PyCharm.
2. Create a new file named `for_loop_practice.py`.
3. Type the following code:

```
# ---------------------------------------------------------
# Big Data Simulation: Transaction Log Scrubbing
# ---------------------------------------------------------

"""
Scenario: We have a list of raw transaction statuses from an API.
We need to normalize the text to uppercase for our Data Warehouse,
and filter out internal test records using the 'continue' keyword.
"""

# 1. EXTRACT: Raw data collection
raw_transactions = ["success", "failed", "INTERNAL", "pending", "success"]
clean_data = []

print("--- Starting Batch Transformation ---")

# 2. TRANSFORM: Iterate over the sequence
for txn in raw_transactions:
```

```
    # Logic Check: Skip internal system tests
    if txn == "INTERNAL":
        print("SKIP: Internal test record ignored.")
        continue # Instantly jumps to the next item in the list

    # Normalization: Convert to uppercase
    normalized_txn = txn.upper()

    # 3. LOAD: Save to our "clean" list
    clean_data.append(normalized_txn)
    print("PROCESSED: " + normalized_txn)

# Final Output
print("-----------------------------------")
print("Transformation Complete.")
print("Final Dataset ready for DB: " + str(clean_data))



Run the code. Output:

--- Starting Batch Transformation ---
PROCESSED: SUCCESS
PROCESSED: FAILED
SKIP: Internal test record ignored.
PROCESSED: PENDING
PROCESSED: SUCCESS
-----------------------------------
Transformation Complete.
Final Dataset ready for DB: ['SUCCESS', 'FAILED', 'PENDING', 'SUCCESS']
```

What happens?

- The `for` loop seamlessly iterates through the `raw_transactions` list one by one.
- When the loop encounters `"INTERNAL"`, the `if` statement triggers the `continue` command. This aborts the current cycle and skips appending it to the `clean_data` list.
- All other valid strings are converted to uppercase using `.upper()` and appended to our final dataset.

Summary

- **`for` loops** are designed for iterating over known sequences (Lists, Ranges, Dictionaries).
- **`range()`** is the standard tool for running a loop a specific number of times.
- **`.items()`** allows you to loop through both Keys and Values in a Dictionary simultaneously.
- **`continue`** skips the rest of the current iteration; **`break`** stops the loop entirely.
- **Data Pipelines depend on `for` loops** to process batches of files, apply transformations to rows of data, and dynamically validate database schemas.

No labels